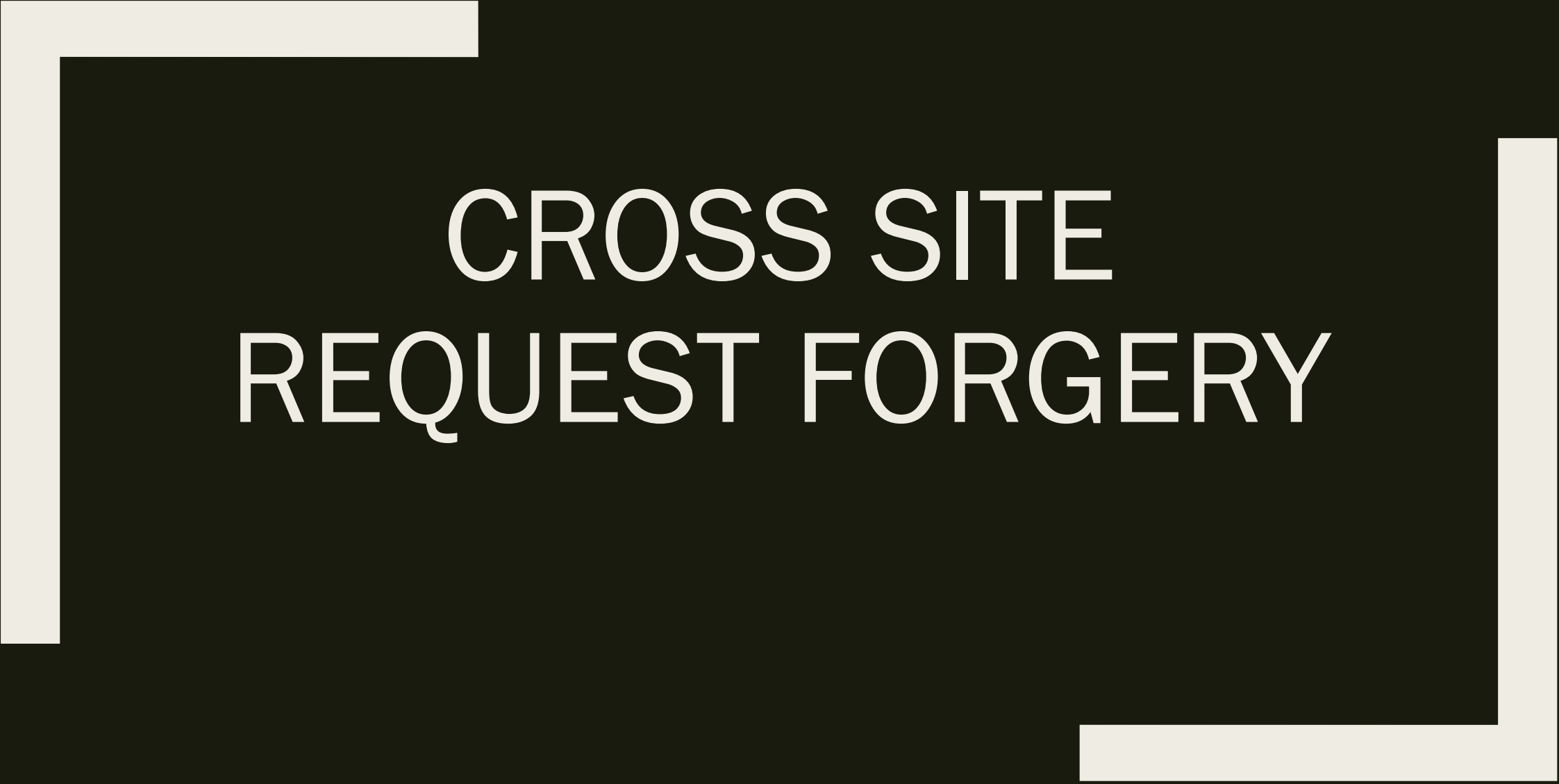


A thick, light-colored L-shaped frame is positioned on the left and bottom edges of the slide, framing the central text.

CROSS SITE REQUEST FORGERY

Cross Site Request Forgery

CSRF (англ. Cross Site Request Forgery — «межсайтовая подделка запроса», также известна как XSRF) — вид атак на посетителей веб-сайтов, использующий недостатки протокола HTTP. Если жертва заходит на сайт, созданный злоумышленником, от её лица тайно отправляется запрос на другой сервер (например, на сервер платёжной системы), осуществляющий некую вредоносную операцию (например, перевод денег на счёт злоумышленника). Для осуществления данной атаки жертва должна быть аутентифицирована на том сервере, на который отправляется запрос, и этот запрос не должен требовать какого-либо подтверждения со стороны пользователя, которое не может быть проигнорировано или подделано атакующим скриптом.

Cross Site Request Forgery

Данный тип атак, вопреки распространённому заблуждению, появился достаточно давно: первые теоретические рассуждения появились в 1988 году, а первые уязвимости были обнаружены в 2000 году.

Основное применение CSRF — вынуждение выполнения каких-либо действий на уязвимом сайте от лица жертвы (изменение пароля, секретного вопроса для восстановления пароля, почты, добавление администратора и т. д.).

Какие это действия – отправка ли сообщений, перевод денег со счёта на счёт или смена паролей – зависят от сайта, но в любом случае эта атака входит в образовательный минимум веб-разработчика.

Злая форма

«Классический» сценарий атаки таков:

- Вася является залогиненным на сайт, допустим, mail.com. У него есть сессия в куках.
- Вася попал на «злую страницу», например хакер пригласил его сделать это письмом или как-то иначе.
- На злой странице находится форма такого вида:

Злая форма

```
<form action="http://mail.com/send" method="POST">
```

```
  <input type="hidden" name="message" value="Сообщение">
```

```
  ...
```

```
</form>
```

Злая форма

- При заходе на злую страницу JavaScript вызывает `form.submit`, отправляя таким образом форму на `mail.com`.
- Сайт `mail.com` проверяет куки, видит, что посетитель авторизован и обрабатывает форму. В данном примере форма предполагает посылку сообщения.

Итог атаки – Вася, зайдя на злую страницу, ненароком отправил письмо от своего имени. Содержимое письма сформировано хакером.

Защита

В примере выше атака использовала слабое звено авторизации.

Куки позволяют сайту `mail.com` проверить, что пришёл именно Вася, но ничего не говорят про данные, которые он отправляет.

Иначе говоря, куки не гарантируют, что форму создал именно Вася. Они только удостоверяют личность, но не данные.

Защита

Типичный способ защиты сайтов – это «секретный ключ» (secret), специальное значение, которое генерируется случайным образом и сохраняется в сессии посетителя. Его знает только сервер, посетителю мы его даже не будем показывать.

Затем на основе ключа генерируется «токен» (token). Токен делается так, чтобы с одной стороны он был отличен от ключа, в частности, может быть много токенов для одного ключа, с другой – чтобы было легко проверить по токену, сгенерирован ли он на основе данного ключа или нет.

Защита

Для каждого токена нужно дополнительное случайное значение, которое называют «соль» salt.

Формула вычисления токена:

`token = salt + ":" + MD5(salt + ":" + secret)`

Защита

Например:

В сессии хранится `secret="abcdef"`, это значение создаётся один раз.

Для нового токена сгенерируем `salt`, например пусть `salt="1234"`.

`token = "1234" + ":" + MD5("1234" + ":" + "abcdef") = "1234:5ad02792a3285252e524ccadeeda3401"`.

Защита

Это значение – с одной стороны, случайное, с другой – имея такой token, мы можем взять его первую часть 1234 в качестве salt и, зная secret, проверить по формуле, верно ли он вычислен.

Не зная secret, невозможно сгенерировать token, который сервер воспримет как правильный.

Далее, токен добавляется в качестве скрытого поля к каждой форме, генерируемой на сервере.

То есть, «честная» форма для отсылки сообщений, созданная на <http://mail.com>, будет выглядеть так:

Защита

```
<form action="http://mail.com/send" method="POST">  
  <input                                type="hidden"                                name="csrf"  
value="1234:5ad02792a3285252e524ccadeeda3401">  
  <textarea name="message">  
    ...  
  </textarea>  
</form>
```

Защита

При её отправке сервер проверит поле `csrf`, удостоверится в правильности токена, и лишь после этого отошлёт сообщение.

«Злая страница» при всём желании не сможет сгенерировать подобную форму, так как не владеет `secret`, и токен будет неверным.

Такой токен также называют «подписью» формы, которая удостоверяет, что форма сгенерирована именно на сервере.

Защита

Требования к токenu:

- ☐ Уникальный токен для каждой операции
- ☐ Действует единожды
- ☐ Имеет размер, устойчивый к подбору
- ☐ Сгенерирован криптографически стойким генератором псевдослучайных

чисел

- ☐ Имеет ограниченное время жизни

Итог

- ❑ CSRF-атака – это когда «злая страница» отправляет форму или запрос на сайт, где посетитель, предположительно, залогинен.
- ❑ Если сайт проверяет только куки, то он такую форму принимает. А делать это не следует, так как её сгенерировал злой хакер.
- ❑ Для защиты от атаки формы, которые генерирует mail.com, подписываются специальным токеном. Можно не все формы, а только те, которые осуществляют действия от имени посетителя, то есть могут служить объектом атаки.
- ❑ Для подписи XMLHttpRequest токен дополнительно записывается в куку. Тогда JavaScript с домена mail.com сможет прочитать её и добавить в заголовок, а сервер – проверить, что заголовок есть и содержит корректный токен.
- ❑ Динамически сгенерированные формы подписываются аналогично: токен из куки добавляется как URL-параметр или дополнительное поле.